

ECE 385
Spring 2025
Final Project

Real-Time QVGA Video Streaming with Filters on FPGA

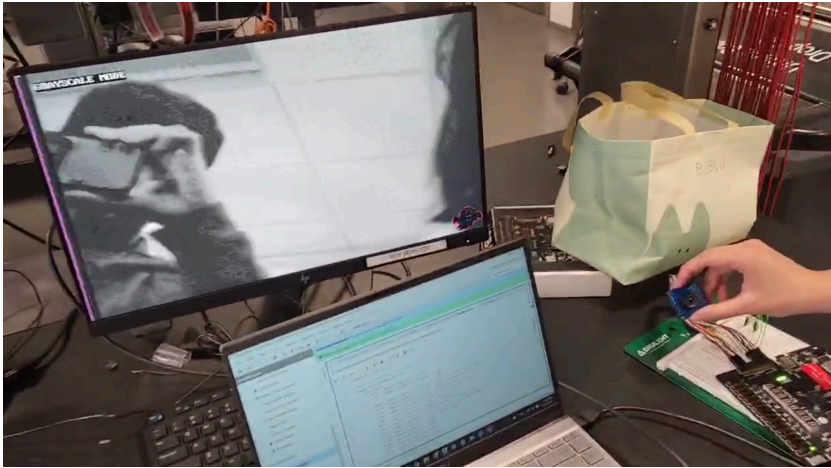
Ayush Barik & Joanna Li
(JH) Friday 15:20
Jinrui Hu

Introduction

When starting this project, we knew we wanted to do something with camera interfacing. The challenge was that we hadn't really learned about cameras or communication protocols like I2C in class, and we had to find a way to make sure our project fit within the limited resources available on the Urbana Board. Unlike other boards, it did not directly have a video input, so we would have to interface the signals correctly if we wanted to obtain a correct HDMI output (which, luckily, we were able to build off the VGA-to-HDMI modules we had used in labs 6 and 7). We decided that we would try to get the camera outputting a working live feed as a baseline, and then make it interactive by adding extra user-controlled effects on top. Though this deviated from our original proposal (which was over-ambitious), we still learned a lot and were able to have a fully functional camera feed in the end.

This final project demonstrated a complete, self-contained system for live digital video processing with filters. Leveraging the principles and components explored in previous labs, our work presents an integrated solution that captures video from an OV7670 camera, performs real-time image manipulation at QVGA resolution, and displays the output on an HDMI monitor. A key design achievement is the system's reliance solely on internal BRAM for all frame buffering and data storage, which was made possible through the use of QVGA (320x240) resolution, thereby eliminating the need for external memory controllers like DDR3.

The developed video pipeline encompasses camera configuration via SCCB, direct capture of the QVGA video stream into BRAM, and a set of user-selectable preset visual modes. These modes, activated by on-board switches, each apply a distinct video effect – such as normal color pass-through, grayscale conversion, or specific color channel suppression (resulting in effects named "Ogre Mode," "Royal Mode," and "Cutesy Mode"). Each mode automatically displays a corresponding textual identifier on screen. Furthermore, the "Grayscale Mode" distinctively features an extremely scary ghost sprite overlay in a fixed screen position at the bottom-right. The processed frames are then scaled and timed for VGA-compatible output, which is subsequently serialized for HDMI display. This project showcases an efficient approach to real-time video within the resource constraints of the FPGA.



(above is a demonstration of the project with the grayscale mode active)

System Design & Implementation

The final project implements a complete FPGA-based system for live QVGA video processing and display. The core functionality involves capturing video from an OV7670 camera, configuring the camera using SCCB, processing the video data with user-selectable filters, and outputting the result to an HDMI monitor. All video buffering and processing are handled using on-chip BRAM, operating at QVGA (320x240) resolution with a 12-bit RGB444 color depth. The processed QVGA stream is then scaled to a 640x480 VGA resolution for display.

When designing this project, we started off with connecting and configuring the camera itself—that was the first step. We decided to go with QVGA output and RGB444, instead of say, VGA and RGB333 (which also would've fit on BRAM, but would take up almost double the space and also appear more jittery in terms of colors). Next, we needed a way to capture each byte of data from the camera and construct a full RGB pixel (12 bits)--this eventually became our `cam_capture` module. The problem of different clock domains—pixel clock and the VGA clock could be resolved by a dual-port BRAM used as a frame buffer—it would capture full frames, as written into by `cam_capture`, to prevent glitches and stuttering on the video output. Lastly, we would read from BRAM in a way that allowed us to display the QVGA output to the full VGA size. We chose to do simple pixel doubling, which is essentially reading each X and Y twice.

Clocking Scheme:

- 24 MHz: Provided to the OV7670 camera module as its input clock.

- 25 MHz: Used as the pixel clock for the VGA timing generation and for driving the display-side logic, including reading from the frame buffer and interfacing with the HDMI transmitter.
- 125 MHz: Required by the `hdmi_tx_0` IP for 5x serialization of the 25 MHz pixel data for TMDS signaling. All resets throughout the system are synchronized to the appropriate clock domains and are derived from a debounced active-high master reset input.

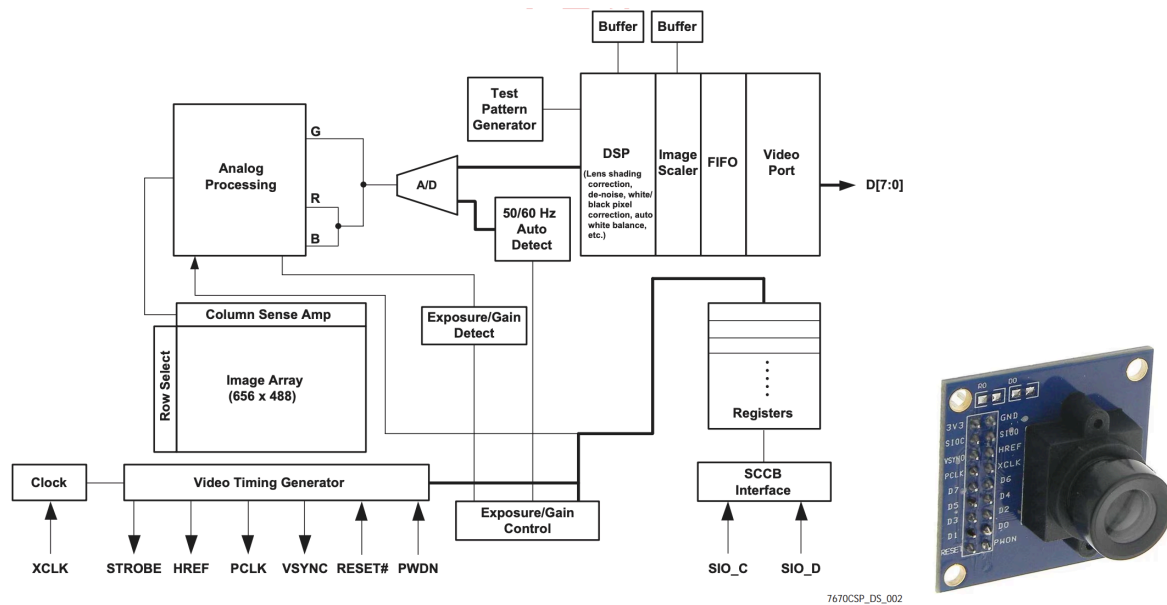
Camera Interface and Configuration:

Our pipeline first starts with the OV7670 camera module. The FPGA interfaces with the camera using standard signals: `cam_pclk` (pixel clock output from camera), `cam_href` (horizontal reference, indicates active line data), `cam_vsync` (vertical sync), and an 8-bit parallel data bus `cam_data`. The FPGA provides the `cam_xclk` (24 MHz) to the camera.

Upon system startup and assertion of the `config_start` signal (debounced switch input), the `camera_config` module initiates communication with the OV7670 camera via the SCCB (Serial Camera Control Bus) protocol. This module, together with the `sccb_controller` module, writes a predefined sequence of configuration values to the camera's internal registers. These values are from `config_rom`, which is sourced from a GitHub online. We did NOT write this ROM.

The configuration sequence sets the camera to output video in QVGA (320x240) resolution and RGB444 format. In RGB444, each pixel's color data is transmitted from the camera in two consecutive 8-bit cycles over `cam_data`, requiring reassembly into a 12-bit (4 bits for Red, 4 bits for Green, 4 bits for Blue) pixel value within our program itself.

The SCCB implementation is a master-only interface, sending write commands to the camera. The SCCB protocol used is a variation where the 9th bit is treated as a "don't care" (or "fake ACK", kind of similar to the ACK bit in I2C), simplifying the master logic.



(to the left is the internal block-diagram of the OV7670 camera)

Video Capture and Frame Buffering:

Pixel Data Capture: The `cam_capture_no_fifo` module monitors `vsync` for frame start/end and `cam_href` for active pixel data on each line. It captures the 8-bit `cam_data` synchronous to `cam_pclk`. Since RGB444 data for one pixel arrives over two `cam_pclk` cycles, this module buffers the first byte (containing, for example, the red component) and then combines it with the second byte (containing green & blue components) to form complete 12-bit RGB444 pixel.

Frame Buffer (BRAM): The captured 12-bit QVGA pixels are written into a dual-port BRAM (`blk_mem_gen_0` instantiated as `frame_buffer` in `module_top`). This BRAM is configured to store one QVGA frame (320x240 pixels, 12 bits each, totaling 76,800 pixels or approximately 112.5 KB).

Write Port: The write port of the BRAM is controlled by the camera capture logic. As pixels are captured and assembled, they are written sequentially into the BRAM. The write address increments for each valid pixel. Write operations are synchronized to `main_clk`. The `bram_wr_en` signal strobes valid writes.

Read Port: The read port of the BRAM is synchronized to the 25 MHz display clock (`clk_25MHz`). It allows the display generation logic to read pixel data for output. The read address (`qvga_addr`) is generated based on the current VGA scanout coordinates, scaled for QVGA.

Video Processing and Effects

Once a frame is available in the BRAM, or on-the-fly as pixels are read for display, several video processing effects can be applied. These effects are selected via on-board switches (grayscale, no_red, no_green, no_blue), which are debounced.

The system supports the following preset visual modes:

Normal Mode: If no filter switches are active, the 12-bit RGB444 pixel data read from BRAM (cam_red, cam_green, cam_blue) is passed through unmodified. The text "NORMAL MODE" is overlaid.

Grayscale Mode: When the grayscale switch is active:

The grayscale_converter module processes the incoming 12-bit RGB pixel (composed of 4 bits for red, 4 bits for green, and 4 bits for blue) to produce a 4-bit grayscale value. This conversion is implemented by first scaling each 4-bit color component to an 8-bit intermediate value. A weighted sum is then calculated using specific bit-shifts on these scaled 8-bit values:

- Red contribution is $(R_scaled \gg 2) + (R_scaled \gg 5) + (R_scaled \gg 6)$.
- Green contribution is $(G_scaled \gg 1) + (G_scaled \gg 4) + (G_scaled \gg 5)$.
- Blue contribution is $(B_scaled \gg 3)$.

The sum of these terms forms an 8-bit intermediate grayscale_value. The final 4-bit gray output is taken from the most significant 4 bits of this grayscale_value (grayscale_value[7:4]). The system then sets the Red, Green, and Blue channels for display to this common 4-bit gray value. The text "GRAYSCALE MODE" is overlaid.

The ghost_example sprite module also becomes active. This module reads pixel data from a dedicated ghost_rom (containing a 64x64 pixel sprite definition) based on the current display coordinates (DrawX, DrawY) relative to the sprite's intended position. A ghost_palette then maps the ROM data to 12-bit RGB values. Pixels marked as transparent in the sprite data are not drawn, allowing

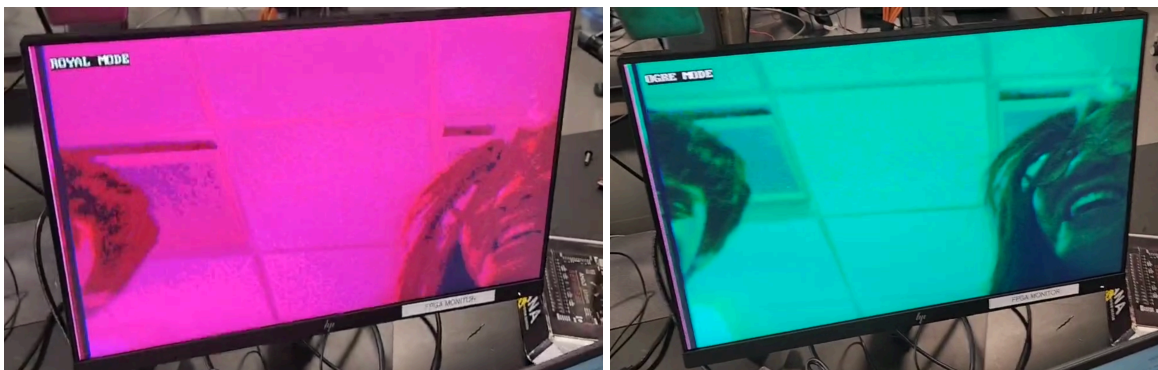
the underlying video to show through. The sprite is displayed at a fixed location (e.g., bottom right of the QVGA area).

Ogre Mode: When the no_red switch is active, the red component is set to 0, while green and blue pass through. The text "OGRE MODE" is overlaid.

Royal Mode: When the no_green switch is active, the green component is set to 0. The text "ROYAL MODE" is overlaid.

Cutesy Mode: When the no_blue switch is active, the blue component is set to 0. The text "CUTESY MODE" is overlaid.

The text_mapper module overlays mode-identifying text (e.g. top-left) in every video mode—except where a grayscale sprite might overlap—by using DrawX/DrawY to pick the current ASCII character and its row, addressing an 8×16 bitmap in font_rom. The ROM's output bit then selects either a white text pixel or a black/transparent background pixel to let the video show through.



(outputs of Royal Mode and Ogre Mode respectively)

VGA Signal Generation and HDMI Output

The system generates a standard 640x480 VGA resolution output, driven by a 25 MHz pixel clock (clk_25MHz). A vga_controller module produces the necessary timing signals and pixel coordinates (DrawX, DrawY).

A key aspect of this stage is the QVGA to VGA scaling. The 320x240 pixel image stored in the BRAM is displayed on the 640x480 VGA canvas by implementing pixel doubling. The DrawX and DrawY coordinates are scaled down (effectively DrawX[9:1], DrawY[9:1]) to address the QVGA frame

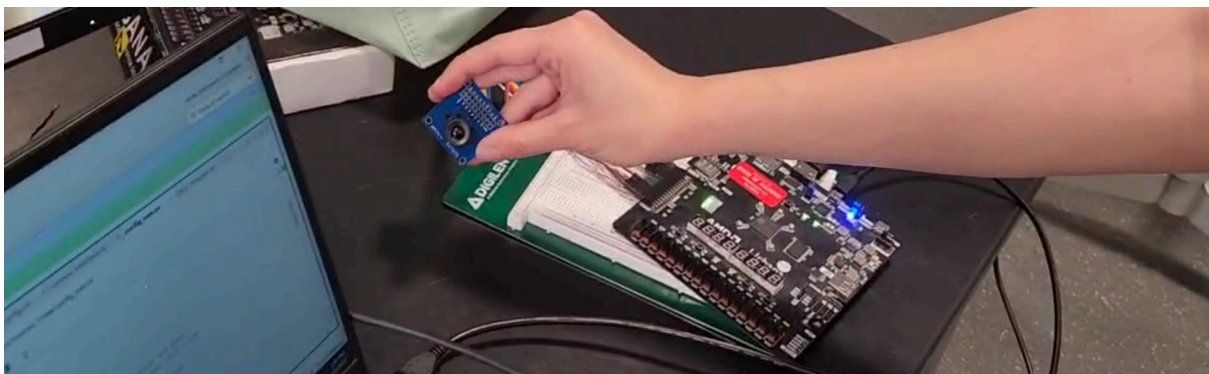
buffer, causing each 12-bit RGB444 pixel from BRAM to occupy a 2x2 area on the final display.

The final 12-bit pixel data sent to the display is determined by an output logic. This logic prioritizes the text overlay, then the sprite (if active in grayscale mode), and finally the (potentially filtered) video data from the scaled BRAM output. This selection occurs on a per-pixel basis, synchronized with `clk_25MHz`. This resulting 12-bit RGB444 video stream, along with the VGA sync signals, is then processed by the Xilinx `hdmi_tx_0` IP core, which handles the TMDS encoding and serialization for output via the HDMI connector, utilizing both the 25 MHz pixel clock and a 125 MHz serialization clock.

User Controls

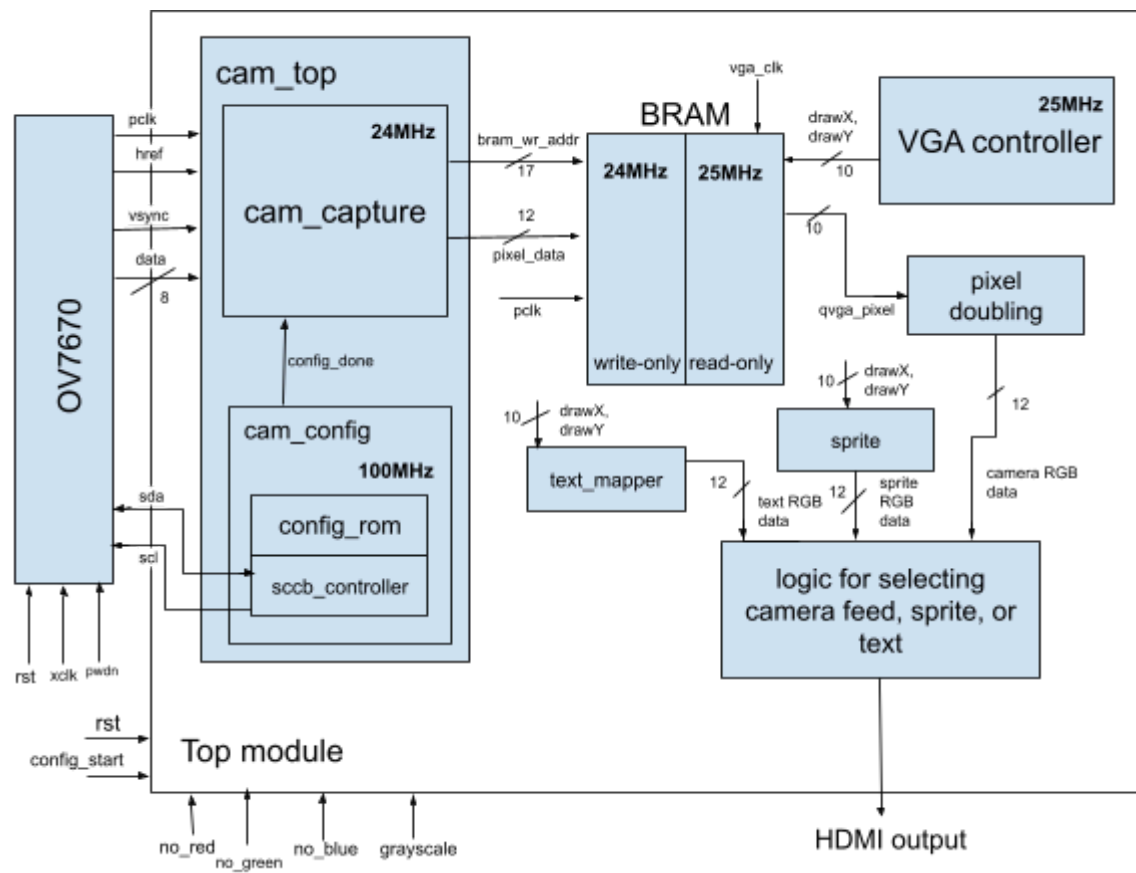
User interaction is primarily through on-board switches:

A `config_start` switch, when toggled, initiates the camera configuration process. Four switches (`grayscale`, `no_red`, `no_green`, `no_blue`) select the active video processing mode.

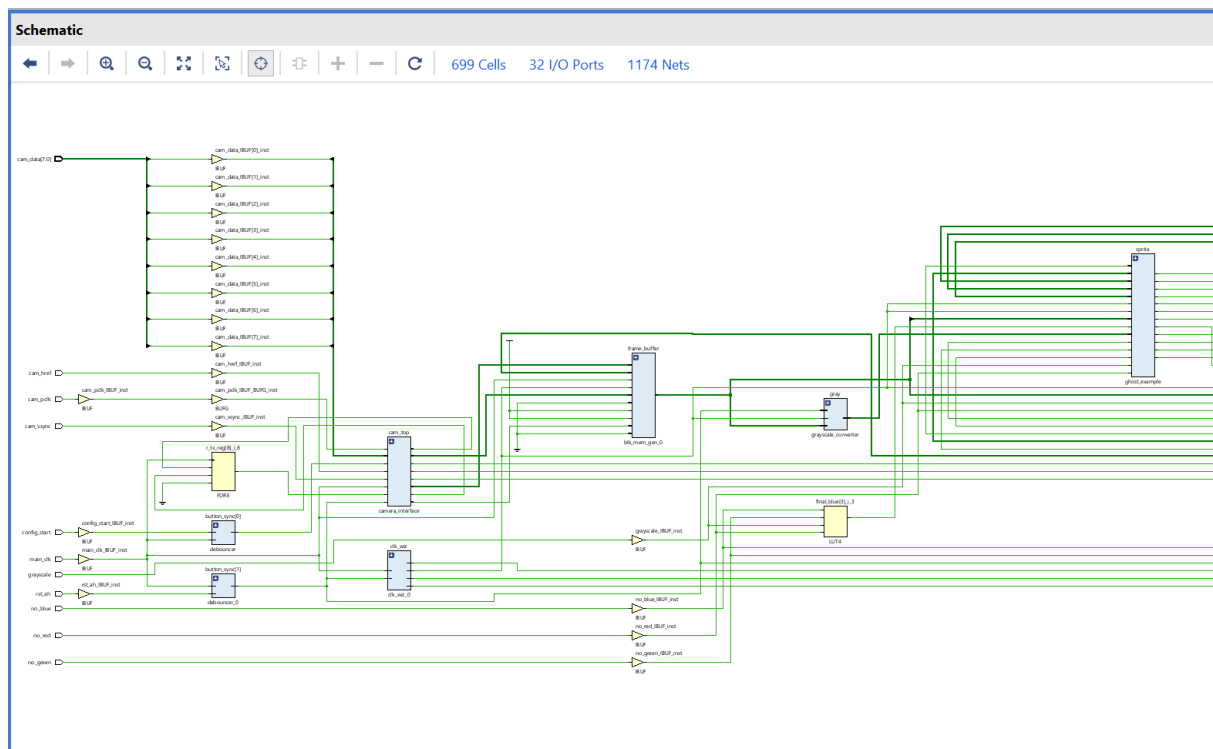


(The hardware setup with the camera held above and wired to the FPGA)

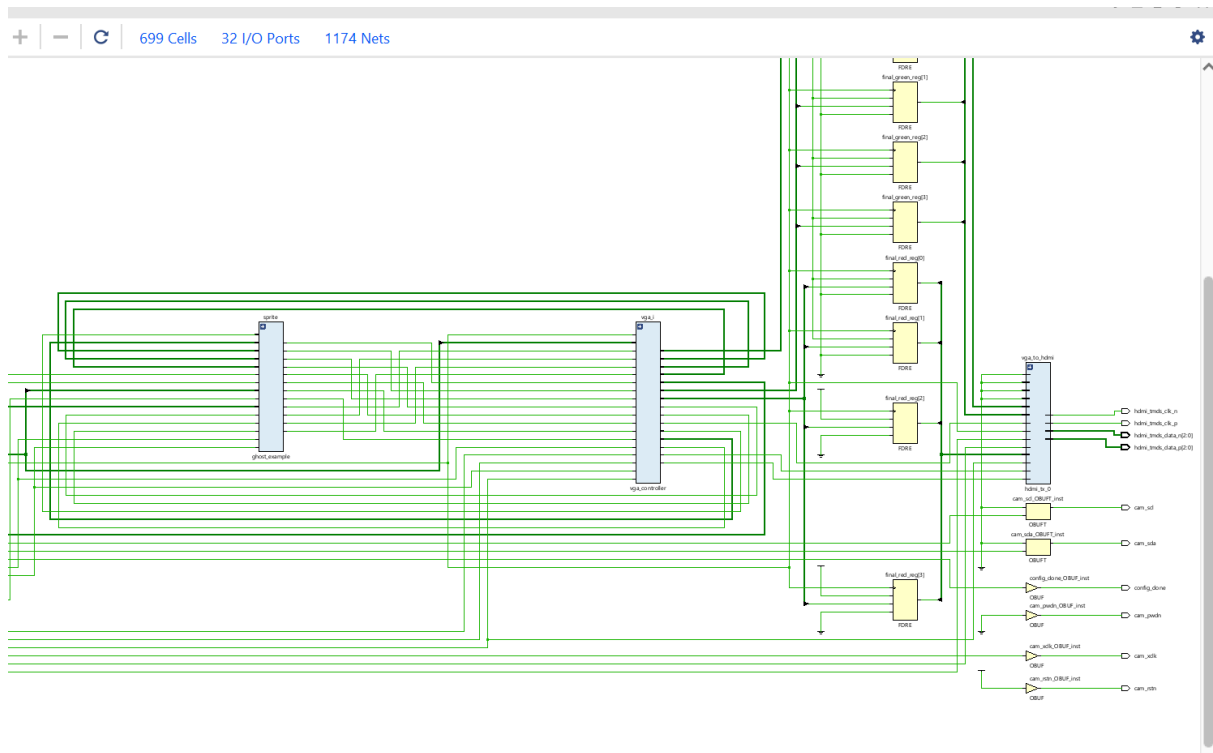
Block Diagram



Schematic (too complicated for clean viewing—refer to block diagram)

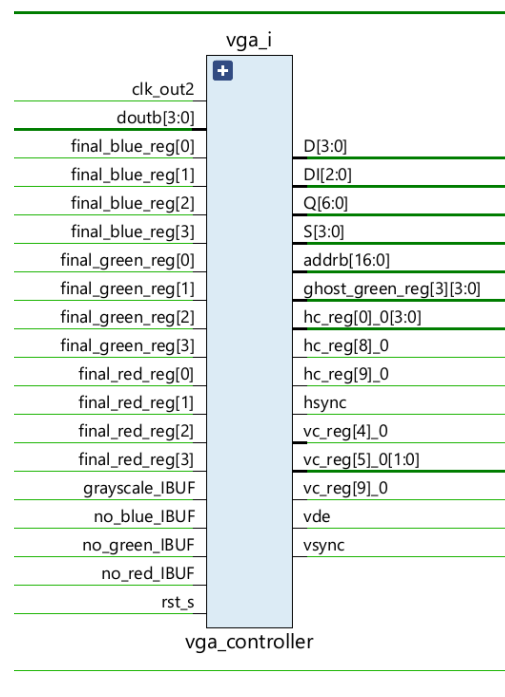
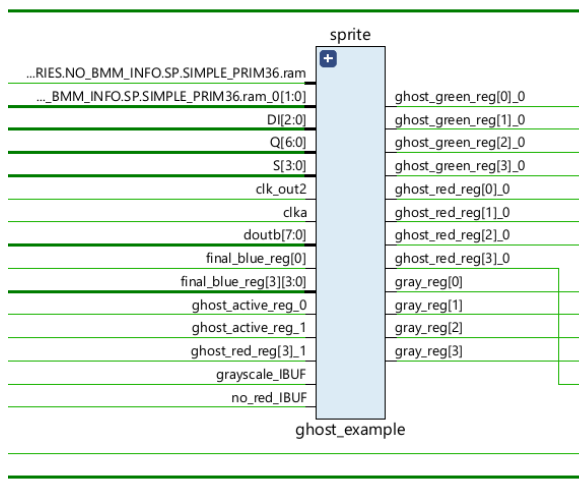
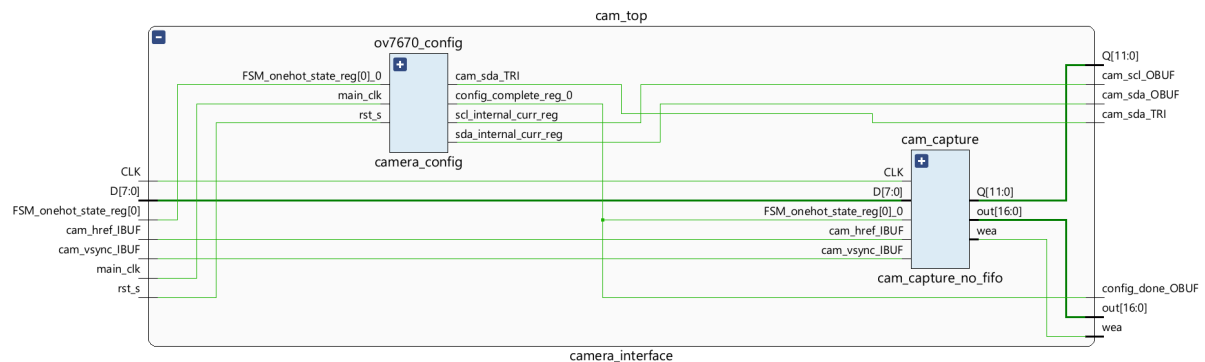


```
(left side← inputs→debouncer→cam_top→frame_buffer→grayscale→sprite)
```



(right side: sprite→vga contrtoller→vga to hdmi→output)

Camera Interface:

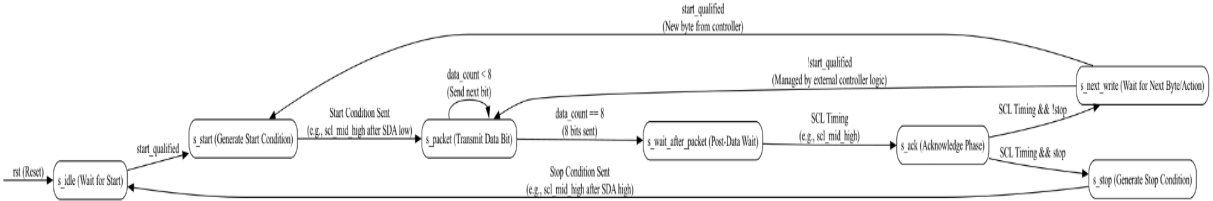


(too big to meaningfully expand)

Important FSMs:

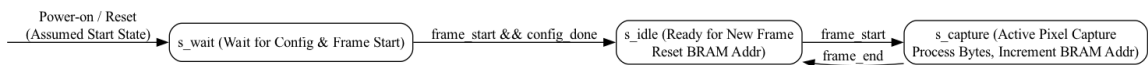
FSM for sccb_controller.sv:

This FSM governs the low-level SCCB communication protocol. It generates precisely timed SCL and SDA signals to serialize data bytes, managing start, bit-wise data transmission, acknowledge/9th-bit cycles, and stop conditions as instructed by higher-level modules.



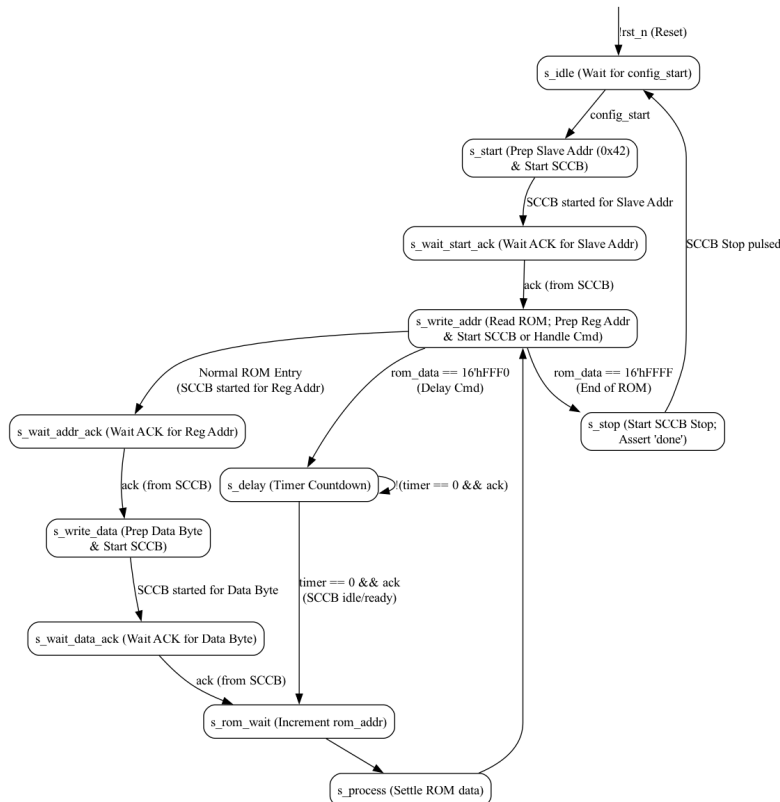
FSM for *cam_capture_no_fifo.sv*:

The *cam_capture_no_fifo* FSM controls the frame-level video capture process. It waits for configuration completion and frame start (*s_wait*), then prepares for a new frame by resetting addressing (*s_idle*), and finally manages active pixel data acquisition and BRAM write strobing throughout the frame (*s_capture*), returning to idle at frame end.



FSM for *camera_config.sv*:

This FSM orchestrates the complete OV7670 camera initialization. Triggered by *config_start*, it sequentially reads configuration entries (register addresses and data values, or special commands like 'delay' or 'end') from an internal ROM. For each entry, it directs the *sccb_controller* module to perform the necessary SCCB byte writes, waiting for completion signals before proceeding to the next ROM command until the entire configuration sequence is done.



Module and Block Design Description

Module descriptions:

Module: debouncer

Inputs: logic Clk (System clock), logic d (Input signal).

Outputs: logic q (Debounced output signal).

Description: Synchronizes the input signal d to Clk using two flip-flops and employs a counter (synthesized for a short stability period) to ensure d is stable before updating the output q, filtering out mechanical switch bounces.

Purpose: To provide clean, glitch-free signals from asynchronous pushbutton/switch inputs for reliable system control.

Module: config_rom

Inputs: logic clk (System clock), logic [7:0] addr (Address input).

Outputs: logic [15:0] dout (16-bit data output).

Description: A Read-Only Memory storing a predefined sequence of 16-bit camera register addresses and data values. Outputs data corresponding to the input addr. Special values indicate end-of-sequence or required delays.

Purpose: Provides the configuration values for initializing the OV7670 camera to QVGA RGB444 mode via the SCCB interface.

Module: sccb_controller

Inputs: logic clk (System clock), logic rst (Reset), logic start (Start transaction), logic stop (Stop transaction), logic [7:0] wr_data (Data to write).

Outputs: logic scl (SCCB clock), inout logic sda (SCCB data), logic ack_received (Transaction acknowledge/9th bit cycle done).

Description: Implements an SCCB master for serial communication. Generates scl based on system clock and SCCB frequency params. Manages SCCB start, data bit serialization (MSB 1st), and stop conditions based on control inputs.

Purpose: Provides the low-level SCCB communication protocol to write data to the OV7670 camera registers.

Module: cam_capture_no_fifo

Inputs: logic pclk (Camera pixel clock), logic vsync (Vertical sync), logic href (Horizontal reference), logic [7:0] cam_data (8-bit camera data), logic config_done (Camera configuration status).

Outputs: logic [16:0] bram_wr_addr (BRAM write address), logic [11:0] pixel_data (12-bit RGB444 pixel), logic write_enable (BRAM write strobe).

Description: Captures video data synchronous to pclk. Monitors vsync and href to identify active pixel data. Reconstructs 12-bit RGB444 pixels from the 8-bit cam_data (received over two pclk cycles per pixel). Generates write addresses and strobes for the BRAM frame buffer.

Purpose: To convert raw camera output into a 12-bit RGB444 pixel stream and write it into the on-chip frame buffer.

Module: grayscale_converter

Inputs: logic clk (Pixel processing clock), logic rst_ah (Reset), logic [3:0] red, green, blue (4-bit per component RGB input).

Outputs: logic [3:0] gray (4-bit grayscale output).

Description: Converts a 12-bit RGB444 pixel to a 4-bit grayscale value. Input R, G, B components are scaled, then combined using a specific bit-shift-based weighted sum ($\text{Gray} = (R \cdot 0.297 + G \cdot 0.594 + B \cdot 0.125)$ approx.), and the upper 4 bits of the 8-bit result form the output.

Purpose: Implements the grayscale video filter effect.

Module: text_mapper

Inputs: logic [9:0] DrawX, DrawY (Display coordinates), logic is_grayscale, no_red, no_green, no_blue (Mode flags).

Outputs: logic [3:0] red, green, blue (RGB output for text).

Description: Generates text overlays for the current video mode at a fixed screen position. Selects an ASCII string based on mode flags, looks up character glyphs from an internal font_rom based on DrawX and DrawY, and outputs corresponding text pixels (foreground/background color).

Purpose: To display status information (current video mode) on the screen.

Module: ghost_palette

Inputs: logic [6:0] index (Color index from sprite ROM).

Outputs: logic [3:0] red, green, blue (RGB color output).

Description: A 128-entry color look-up table (LUT) providing 12-bit RGB444 color values based on a 7-bit input index.

Purpose: Maps color indices from the ghost_rom to 12-bit RGB values for sprite rendering.

Module: ghost_example

Inputs: logic vga_clk (Pixel clock), logic [9:0] DrawX, DrawY (Display coordinates), logic blank (Display enable).

Outputs: logic [3:0] ghost_red, green, blue (RGB for sprite), logic ghost_active (Sprite pixel is active and not transparent).

Description: Renders a 64x64 sprite at a fixed screen position (bottom-right of the 640x480 display). It reads pixel data from an instantiated Xilinx Block Memory Generator IP (blk_mem_gen_2 named ghost_rom), which stores the sprite's pixel indices. These indices are then converted to 12-bit RGB using an internal ghost_palette. Handles transparency based on specific palette indices. ghost_active is asserted for visible sprite pixels.

Purpose: To overlay a predefined sprite image onto the video, specifically in "Grayscale Mode."

Module: vga_controller

Inputs: logic pixel_clk (25MHz), logic reset (Active high).

Outputs: logic hs (HSync), logic vs (VSync), logic active_nblank (VDE), logic [9:0] drawX, drawY (Pixel coordinates).

Description: Generates standard VGA timing signals (HSync, VSync, VDE) and pixel coordinates for a 640x480 display resolution based on the input pixel_clk. (Functionally similar to the one used in previous ECE 385 labs).

Purpose: Provides essential timing synchronization and pixel coordinates for the video display pipeline.

Module: camera_config

Inputs: logic sys_clk (System clock), logic rst_n (Reset), logic config_start (Initiate configuration).

Outputs: logic done (Configuration complete), logic scl (SCCB clock), logic sda (SCCB data).

Description: Manages the OV7670 camera configuration. On config_start, it reads address/data pairs from an internal config_rom and uses an sccb_controller instance to write these to the camera. An FSM controls the sequence, handling ROM entries for delays and end-of-sequence.

Purpose: Automates the initialization of the OV7670 camera to the required operational mode (QVGA RGB444).

Module: camera_interface

Inputs: logic sys_clk, logic rst_n, logic cam_pclk, cam_vsync, cam_href, logic [7:0] cam_data, logic cfg_init.

Outputs: logic o_scl, o_sda (SCCB), logic cfg_done, logic [16:0] bram_wr_addr, logic [11:0] bram_wr_data, logic bram_wr_en.

Description: Integrates camera configuration and video capture. Instantiates camera_config to initialize the OV7670 via SCCB. Instantiates cam_capture_no_fifo to process raw camera data and output 12-bit RGB444 pixels with BRAM write controls.

Purpose: Provides a unified interface for controlling the camera and accessing its captured pixel stream for frame buffering.

Module: module_top

Inputs: logic main_clk (100MHz), logic rst_ah, config_start (User controls), grayscale, no_red, no_green, no_blue (Filter switches), camera interface signals (cam_pclk, cam_href, cam_vsync, cam_data).

Outputs: logic cam_xclk (24MHz), cam_scl, cam_sda, config_done, HDMI TMDS signals.

Description: The top-level module integrating all system components. It instantiates clk_wiz_0 for clock generation, debouncer for user inputs, camera_interface for camera handling and pixel capture, blk_mem_gen_0 for the BRAM frame buffer, vga_controller for display timing, video processing modules (grayscale_converter, text_mapper, ghost_example), and hdmi_tx_0 for HDMI output. Implements logic for QVGA-to-VGA scaling and multiplexing video, text, and sprite data.

Purpose: To connect and coordinate all functional blocks, realizing the complete live video processing pipeline.

Key Instantiated IP Cores

Component: Clocking Wizard (clk_wiz_0)

Inputs: main_clk (100MHz), rst_s (Reset).

Outputs: 24MHz (cam_xclk), 25MHz (clk_25MHz for VGA/pixel logic), 125MHz (for HDMI serialization), locked status.

Description: Xilinx IP core configured to generate required system clocks from the main 100MHz input.

Purpose: Provides synchronized clock signals for various parts of the system.

Component: Block Memory Generator (blk_mem_gen_0 as frame_buffer)

Port A (Write): Clock: main_clk, Address: bram_wr_addr ([16:0]), Data In: bram_wr_data ([11:0]), Enable: bram_wr_en.

Port B (Read): Clock: clk_25MHz, Address: qvga_addr (derived), Data Out: qvga_pixel ([11:0]).

Description: Xilinx IP configured as a true dual-port BRAM for QVGA frame storage (320x240 pixels, 12-bit RGB444).

Purpose: Serves as the on-chip frame buffer, allowing simultaneous camera writes and display reads.

Component: HDMI Transmitter (hdmi_tx_0 as vga_to_hdmi)

Inputs: 12-bit RGB pixel data (final_red, green, blue), VGA syncs (hsync, vsync, vde), pix_clk (25MHz), pix_clkx5 (125MHz), rst_s.

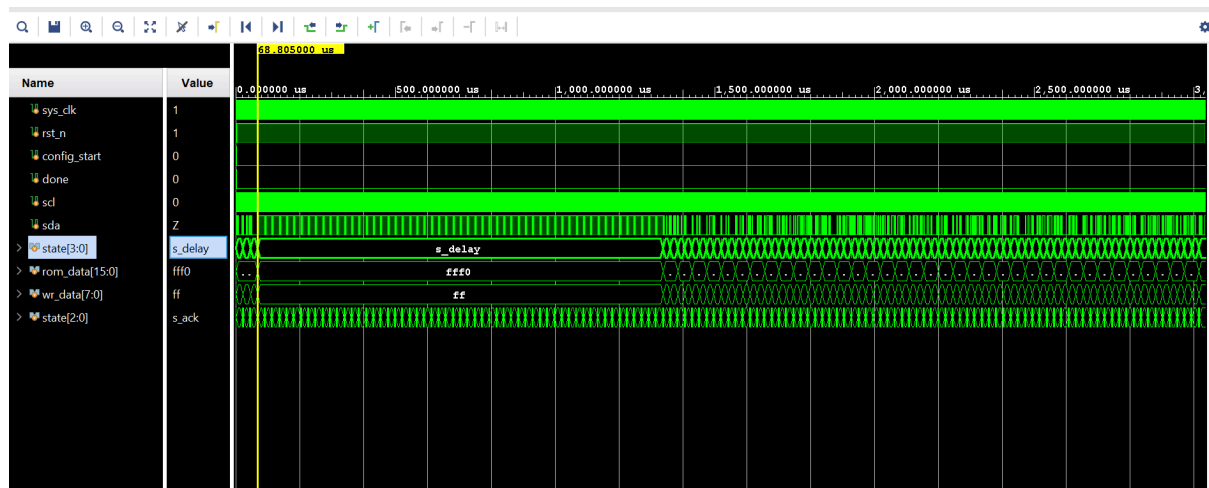
Outputs: Differential TMDS clock and data signals for HDMI.

Description: Xilinx IP core that converts parallel video data and sync signals into TMDS format for HDMI output.

Purpose: Drives the HDMI display with the processed video stream.

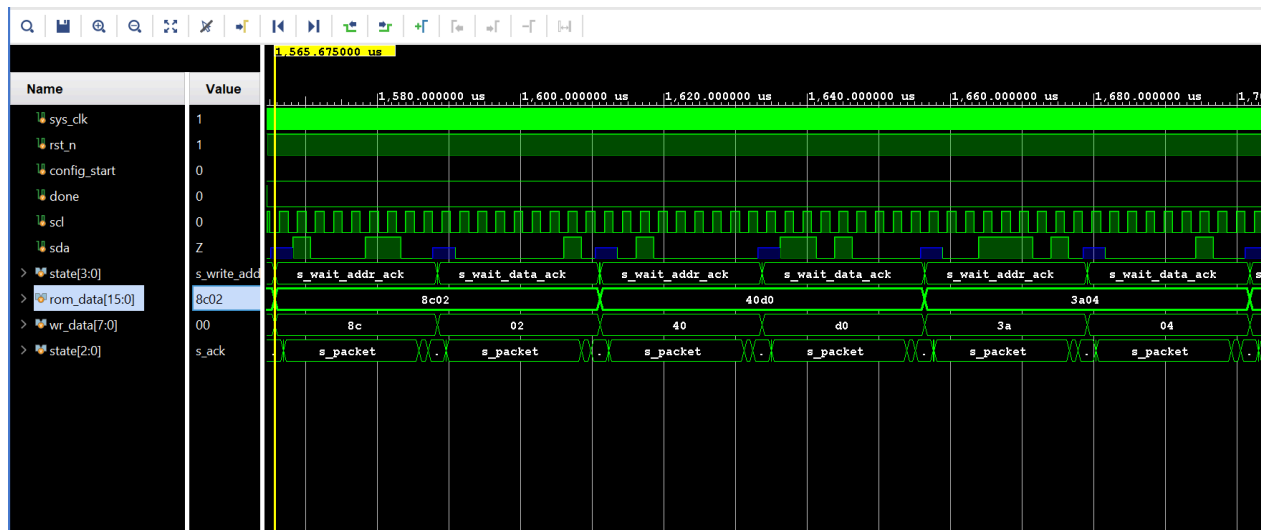
Simulations

Camera_config/sccb_controller:



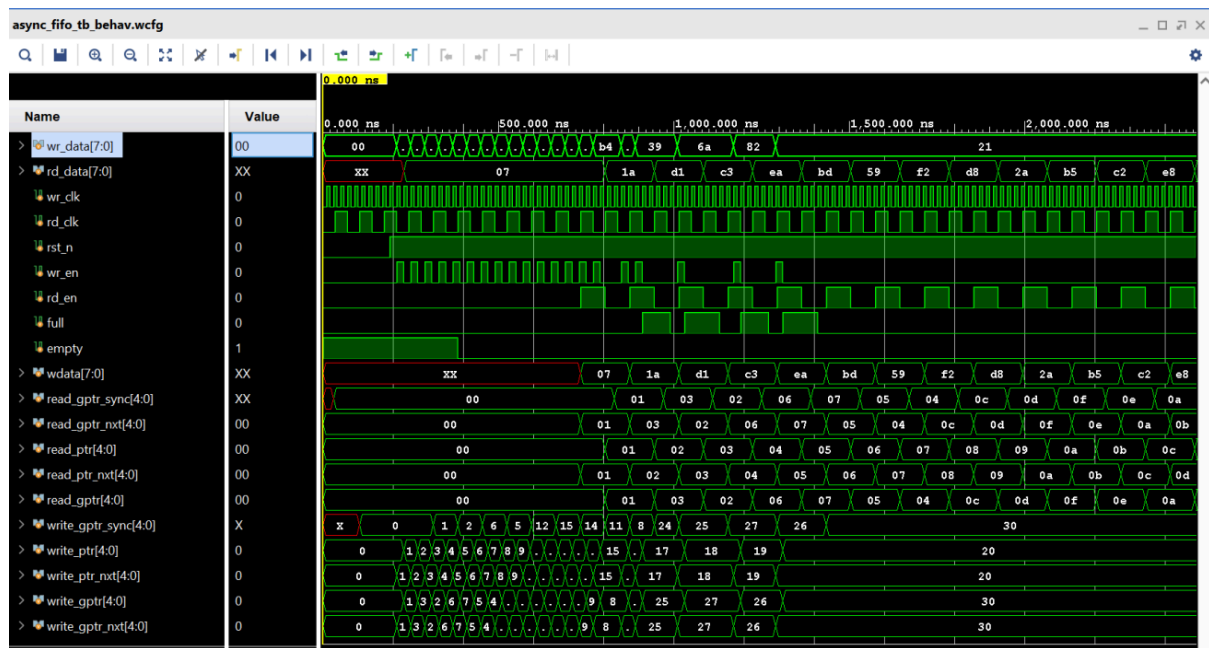
Though we initially started by testing sccb_controller separately, it was also necessary to run simulations for the camera configuration module as a whole (as it involved a separate FSM for combining the SCCB protocol with the camera ROM). In the above waveform, notable things include the fact that there is a delay present in the ROM, which follows directly after the reset command,

16'h1280. This delay allows for a couple of milliseconds in which the camera is able to “settle” before beginning actual configuration, and also allows us to not have to account for time between the start input being received and the time when SIOD drops and configuration begins. This delay is implemented using a simple counter in the FSM—when it reaches zero and when the middle of the next low SIOC cycle arrives, we begin the next write.



Above is a zoomed in shot of the writes occurring after the delay ends. The upper state[3:0] is of the configuration states, while the bottom state[2:0] is the SCCB states. When the SCCB is in s_packet, the 8 bits in the current byte (wr_data) are being transmitted. As each byte of data is being transmitted, the s_wait_addr_ack and s_wait_data_ack states in the configuration module make sure that the ROM address is not incremented / the next byte isn't sent until the current byte is done transmitting (aka when we see the ninth “don't care” bit indicating successful transmission). In this way, we send the address and data bytes of rom_data sequentially through the I2C protocol without losing data.

Asynchronous FIFO:



Though we ended up not using the FIFO in the final design (we were running out of time and needed to reduce complexity to ensure a working design), we were still able to implement a working one. The following source was heavily referenced: <https://vlsiverify.com/verilog/verilog-codes/asynchronous-fifo/>; however, we used one module instead of several. The key aspect of the asynchronous FIFO is that the read and write pointers are synchronized to different clocks. As a result, a 2-FF synchronizer is used to pass the two pointers. Also, we are passing gray-coded pointers into the synchronizer, as this solves the problem of metastability that comes with binary-coded pointers. We are also checking full and empty conditions directly using the received gray-code pointers, according to the formula in the article (shown below):

```
wfull = (g_wptr_next == {~g_rptr_sync[PTR_WIDTH:PTR_WIDTH-1],
g_rptr_sync[PTR_WIDTH-2:0]});
```

```
rempty = (g_wptr_sync == g_rptr_next);
```

In the simulation, we can clearly see that the two clock domains for read and write are operating at different speeds. The upper wr_data[7:0] is the inputted data, while the bottom wdata[7:0] represents what the read side should be seeing and thus outputting (the queue that wr_data is pushed onto).

We also built off the testbench code in the article and converted it to SystemVerilog. Using this testbench, we were able to check that wr_data and rd_data were equivalent when rd_data is reading. The output, as well as the code used to generate this output, are shown below:

```

Time = 1016000: PASS: Data = d1
Time = 1156000: PASS: Data = c3
Time = 1296000: PASS: Data = ea
Time = 1436000: PASS: Data = bd
Time = 1576000: PASS: Data = 59
Time = 1716000: PASS: Data = f2
Time = 1856000: PASS: Data = d8
Time = 1996000: PASS: Data = 2a
Time = 2136000: PASS: Data = b5
Time = 2276000: PASS: Data = c2
Time = 2416000: PASS: Data = e8
Time = 2556000: PASS: Data = 0f
Time = 2696000: PASS: Data = b4
Time = 2836000: PASS: Data = c0
Time = 2976000: PASS: Data = 39
Time = 3116000: PASS: Data = 6a
Time = 3256000: PASS: Data = 82

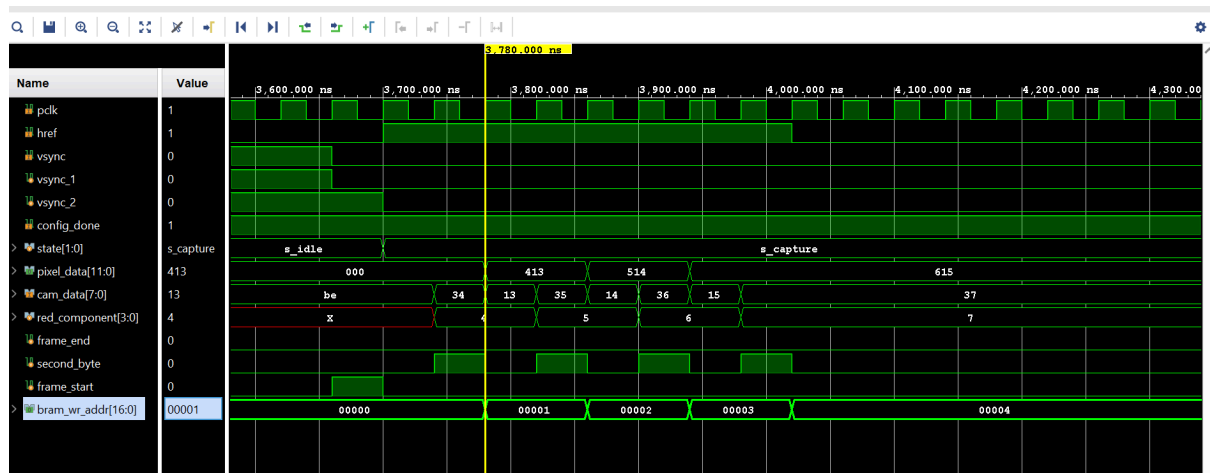
// read logic
initial begin
    rd_clk = 0;
    rd_en = 0;

    repeat(10) @(posedge rd_clk);
    // rst_n is already deasserted by now

    repeat(2) begin
        for (int i = 0; i < 30; i++) begin
            @(posedge rd_clk);
            rd_en = !empty && (i % 2 == 0); // read every other cycle
            if (rd_en) begin
                wdata = wdata_q.pop_front();
                #1; // wait for BRAM delay
                if (rd_data !== wdata)
                    $error("Time = %0t: FAIL: Expected = %h, Got = %h", $time, wdata, rd_data);
                else
                    $display("Time = %0t: PASS: Data = %h", $time, rd_data);
            end
        end
        #50;
    end
end

```


Camera Capture Module (no FIFO)



However, as mentioned before, we removed the use of the FIFO to simplify the overall process. Given that we already implemented a frame buffer in BRAM, we knew we wouldn't experience too much of a drop in quality. The camera capture module was used to concatenate bits from 2 bytes of camera data into 1 12-bit RGB color output, and also generate a bram address for this data to be written to. In the above simulation, `cam_data[7:0]` is the inputted camera data. `Red_component[3:0]` contains the bottom 4 bits from the first byte, which is then combined with the second byte to get the output `pixel_data[11:0]`, which is equal to $\{\text{red_component}, \text{cam_data}[7:0]\}$. We can clearly see this in the waveform: when `frame_start` goes high, we transition into the state `s_capture`. When the input data `8'h34` is received, `red_component` stores the bottom 4 bits, `4'h4`. When the next `cam_data` byte, `8'h13`, is received, the output data is ready: `12'h413`. As soon as this data is ready, `bram_wr_addr` increments. We don't update `red_component` until receiving the next byte. This continues for as long as the camera runs!

Design Resources and statistics

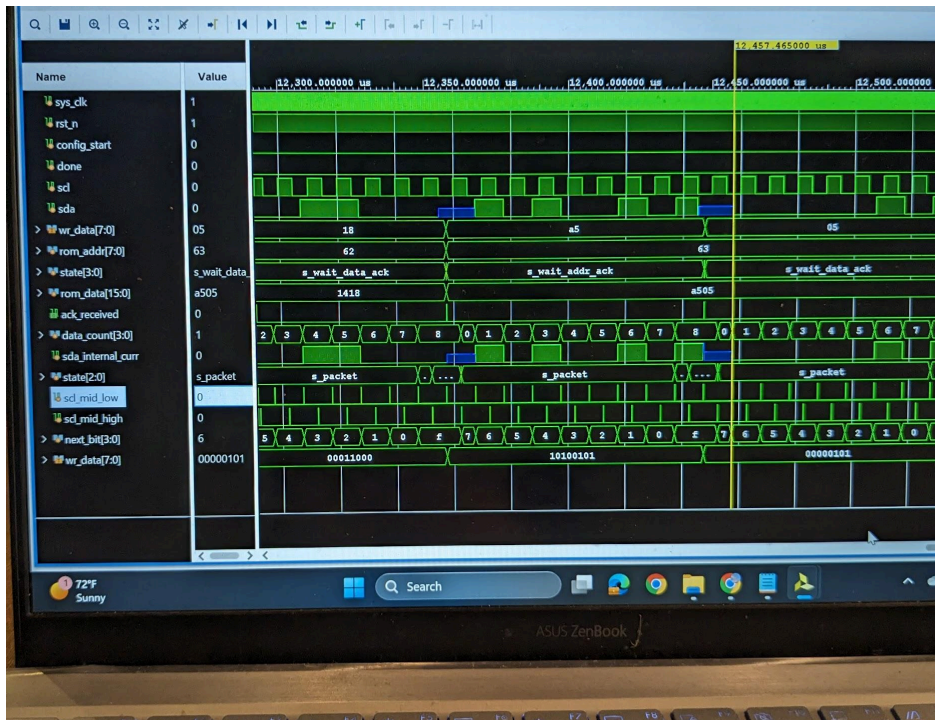
LUT	813
DSP	0
Memory (BRAM)	28
Flip-Flop	309
Latches	0

Frequency	139 MHz
Static Power	0.074 W
Dynamic Power	0.276 W
Total Power	0.35 W

Debugging, Test-Benching, and Struggles

It was an extreme challenge to get the output feed fully operational (40 or so hours of work), as what worked in simulation might not work in reality. The biggest hurdle to get over was camera configuration, where we experienced the disparity between simulation and hardware in full force. Despite the SCCB protocol looking functional in simulation, we were getting a blank screen on the monitor. There were a number of small issues with timing, state changes, and inferred latches that would cause key signals to get optimized out in synthesis—this resulted in the “configuration done” flag never going high. Even after getting `config_done` to show up as an LED on the board, we were still initially seeing no output, and debug cores could only sample a small portion of the configuration sequence. One of our biggest problems was that the SCCB timing was just slightly off. SDA should be changing in the middle of an SCL low, but the 9th “fake ACK” bit, where the slave line was released (high Z), was happening on an odd state transition that caused it to take up more clock cycles than it should. Debugging this, along with other things, took up a majority of our work time, as if the configuration didn’t work, we wouldn’t be able to verify if anything else was working.

Below is a picture of when our 9th bit was not correctly working in the `config_module` (upper SDA waveform). It was transitioning at the falling edge of SCL, even though the SDA of the actual `sccb_controller` (lower SDA waveform) was changing correctly.



In addition, even after getting output, we were initially getting a really strange display: the screen was mostly red, and there were black bars – some flickering, some static in diagonal patterns across the screen (unfortunately, we don't have a picture of this, as my (Joanna's) phone was broken during the time that I was testing this.) However, the flickering showed that the camera was light-sensitive, which meant that it was functional—then, it was either configured wrong, or the camera capture→BRAM pipeline was incorrect. After feeding a pure green input into the capture module, I realized that the issue was in the capture→BRAM. The monitor was displaying the green, which was no longer flickering, but the solid black bars persisted, showing that there were gaps in BRAM storage. The problem was in how we were incrementing the `bram_wr_addr` in the capture module—it was only conditionally incrementing, causing it to miss data at regular intervals.

We were also getting a camera output with no blue whatsoever (all pink and green). We figured out that we were incorrectly combining the bottom 4 bits from the first byte of camera data with the second byte (containing the data for green and blue).

Lastly, it was a struggle to get the PMOD pins secured. The jumper wires that we purchased happened to be just slightly too thin for the ports on the board,

and they kept slipping out if the camera was even slightly adjusted. This also occurred slightly during our demo, but if the pins are held in place, there was no issue with the output. This probably could've been avoided if we had attached the pins to the board ports with tape or something, but since we wanted ease of transport/disassembly, we chose not to.



We also had unfortunate instances like this camera pin getting bent beyond saving... In addition, we also borrowed a camera from a CA that we spent hours debugging before realizing that the camera module itself was broken..

Future Areas of Exploration + Ideas That Didn't Make it

A simple next step would be to include the FIFO in the final design—though we were able to obtain a camera output with minimal if not zero visual jittering, perhaps a FIFO would have stabilised the video output even more, eliminating the possibility of corrupted frames or garbage video data.

Our initial plan was to expand the camera to include hand tracking and audio output, and maybe even a rhythm game extension. Due to lack of time, these ideas were not realized. However, skin (or other material) detection would be a very reasonable extension—our original idea was to just use normalized RGB values, and if the RGB values detected by the camera fell within a certain range, to recognize it as skin. Alternatively it would've also been doable to wrap bright colored fabric/some other material around the finger for better ease of detection (for example, a bright pink is not as likely to blend into background values). We would have divided the screen into sections, with each section representing a key on a piano. If a user's finger was detected in a certain region, it would output the corresponding audio.

Audio generation would likely be using the Karplus Strong String Algorithm for a sound that mimics an actual piano sound, in which a waveform is repeatedly outputted, fed into a delay line, filtered, and mixed in again with the output. Alternatively, we could combine sine/square waves for a desired output. This would all be outputted using the FPGA's audio out port and amplified using a speaker. There would likely be some conversion to PWM or the like.

Many other project extensions could be derived from the camera. One of which would be our rhythm game idea—some playable game that involves movement in real life by the user. Even more complex would be a game that detects full-body movement, which may not be within the scope of this class but is an interesting direction to explore.

Conclusion

This project successfully culminated in a fully operational FPGA-based live video processing system. The implemented design interfaces with an OV7670 camera, performs real-time QVGA (320x240) video processing using only on-chip BRAM, and outputs to an HDMI monitor. Key features include SCCB camera configuration, efficient frame buffering, and several user-selectable preset visual modes – encompassing grayscale conversion, color channel suppression, and integrated text/sprite overlays. Overcoming challenges in system integration and timing provided valuable practical experience in FPGA development and real-time video manipulation, effectively demonstrating the application of core ECE 385 concepts as well as new concepts.

Reflecting on ECE 385, the course effectively introduced SystemVerilog fundamentals and fostered strong collaboration through consistent partner-based labs - a unique aspect at UIUC. While the extensive documentation was sometimes lengthy, the significant reading load did ensure deeper understanding. Though we would say documentation was unclear at times. The lab difficulty was also quite varied - we would say lab 2 & 7 were the hardest, while lab 4 was absolute pain to debug and was the only lab we didn't successfully complete (not knowing how to write testbenches at the time). Quizzes were stressful at times as well but were not too difficult usually. Overall, the course experience was good!

References

We utilized a wide variety of resources to assist us in our project. Some were more helpful than others, some of the concepts never made it into the design but regardless we would like to thank everyone who was behind these resources. Without them our project would have been much more difficult. We would also like to thank Prof. Cheng and all ECE385 CAs & TAs who assisted us.

- VLSIVerify. (n.d.). *Asynchronous FIFO Verilog Code*. Retrieved May 1, 2025, from <https://vlsiverify.com/verilog/verilog-codes/asynchronous-fifo/>
- OmniVision Technologies, Inc. (2006, August 21). *OV7670/OV7171 CMOS VGA (640x480) CAMERACHIPTM with OmniPixel® Technology* (Version 1.4). Retrieved May 3, 2025, from https://web.mit.edu/6.111/www/f2016/tools/OV7670_2006.pdf
- Sheth, A. (amsheth). (n.d.). *Image to COE*. GitHub repository. Retrieved May 3, 2025, from https://github.com/amsheth/Image_to_COE
- Element14 Community. (n.d.). *Security Camera #3 : Interfacing with OV7670 Camera*. Retrieved April 27, 2025, from <https://community.element14.com/challenges-projects/design-challenges/summer-of-fpga/b/blog/posts/security-camera-3-interfacing-with-ov7670-camera>
- NXP Semiconductors. (2021, October 1). *I2C-bus specification and user manual* (Rev. 7.0 - UM10204). Retrieved May 1, 2025, from <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- Texas Instruments. (2023, February). *A Basic Guide to I2C* (Application Report SBAA565). Retrieved April 27, 2025, from <https://www.ti.com/lit/an/sbaa565/sbaa565.pdf>
- CircuitDen. (n.d.). *FPGA Implementation of I2C Protocol (Master & Slave) in Verilog with Code*. Retrieved May 1, 2025, from <https://www.circuitden.com/blog/21>
- Weston, B. (westonb). (n.d.). *OV7670-Verilog (OV7670_config_rom.v)*. GitHub repository. Retrieved April 28, 2025, from https://github.com/westonb/OV7670-Verilog/blob/master/src/OV7670_config_rom.v
- Varola, I. (ibrahimvarola). (n.d.). *OV7670-SCCBMaster (SCCBMaster.vhd)*. GitHub repository. Retrieved April 29, 2025, 2025, from <https://github.com/ibrahimvarola/OV7670-SCCBMaster/blob/main/english/hdl/SCCBMaster.vhd>